FactGate: A Task-Scoped Data Gateway for AI Agents with Cumulative Exposure Accounting

Minmin Wu

Abstract—Per-query authorization does not compose across an adaptive agent's pages, retries, rewrites, and delegated subtasks, and row or query counters are order-dependent. FactGate is a release layer for immutable published data products, reporting relations and document collections alike, that turns one approved task into a cumulative contract: three set-valued budgets over declared identities (result, positive-output dependency, and outcome facts) shared by the root task and descendants. Exact set novelty against the root-family ledger is settled atomically on one PostgreSQL snapshot before any result becomes retrievable. The guarantee is bounded *release*: within one root family and publication epoch, at most the approved number of distinct declared facts of each kind becomes available, whatever the admitted sequence; inference from answered or refused predicates is not bounded. Accounting is exact under declared rules over a closed fragment (connected equijoins, grouping, bounded unions, nested Views, derived-cell arithmetic) that admits 55% of 40 held-out LLM-written statements (70% of a development set adapted during construction). Ordinals, bitmaps, and a Merkle-radix set keep novelty exact at 10^7 facts. A single-host campaign (9,393 samples) charges every scripted decomposition once, keeps an adaptive trace within its budgets, and costs 21.8–144.0$\times$ a direct query; settlement takes a median 163 ms against a million-fact history at zero overlap. Because the contract is stated over Fact sets, not SQL, the same gateway governs exact vector and keyword retrieval over a published document collection: on fresh deployments both return the hit sets of direct execution and every scripted extraction stops at its budget.

Index Terms—data exposure, query auditing, provenance, adaptive workloads, cumulative budget, dependency footprint, outcome accounting

1 Introduction

Database authorization asks, "Can this query execute?" An adaptive agent adds, "How much policy-defined exposure has this task acquired?" Within one approved task, an agent may split projections, overlap pages, retry, rewrite, and delegate (each child grant human-approved) while individual queries remain legal. Per-query authorization and row limits do not state the root family's distinct cumulative exposure. FactGate accounts that contract-defined quantity, not the agent's knowledge. Data-flow control (DFC) asks whether a SQL flow is allowed [1], [2]; given authorization and any DFC check, FactGate asks whether the next allowed execution fits the root's cumulative contract.

• *M. Wu is with China Telecom Global Limited. E-mail: wuminmin@chinatelecomglobal.com.*

Query count ignores answer size, row count makes a one-row aggregate over millions of source rows look cheap, and bytes depend on encoding. Query pricing develops anti-arbitrage objectives [3]–[5] and provenance explains where results came from [6], [7]; FactGate asks how human authorization can create a stateful release budget conserved across adaptive querying and delegation. Enterprise operational analytics commonly serves scheduled ETL outputs rather than OLTP primaries, so FactGate compiles each completed cut into an immutable publication and bounds the governed release of declared facts within one reporting epoch, not freshness over a changing database.

Large enterprises operate warehouses, reporting marts, document repositories and knowledge bases, Data Catalogs, identity directories, and approval workflows, but none turns one approved purpose into an enforceable contract across an agent's queries, retries, pages, and delegated subtasks; an agent allowed to search one department's policy library can copy much of it through individually legitimate searches. Marketplace-integrated agents close the discovery-and-request half of this gap (Data Product MCP [8]–[10]), but after approval, per-query checks still do not state how much governed data the task has released. FactGate starts at that boundary as a brownfield-compatible egress layer at the governed serving tier. The reporting pipeline supplies immutable publications, the Catalog supplies Products and attested keys, and identity/workflow systems supply subject, purpose, and approval. FactGate compiles these into a signed task contract at the database boundary and never exposes the raw lake to the agent.

FactGate budgets the root task in three set-valued dimensions. *Accounted result exposure* identifies selected result values (wire label *release*); accounting can precede availability. The *positive-output dependency footprint* identifies the base facts the declared rules read—neither minimal causes nor all physical reads—so a one-cell aggregate can expose large input. *Query outcome* combines canonical predicate atoms with one normalized query/result composite. Existing members add no novelty: related thresholds reuse shared atoms but charge different literals/composites; exact replay is free.

The system follows a withhold-before-settlement discipline (`reserve -> replay lookup / execute+stream -> derive -> settle -> publish`; Fig. 1). The visible query and ordinal companion share one frozen PostgreSQL snapshot and remain withheld while one Control transaction computes exact set novelty,

checks all limits, advances the root head, and commits audit, receipt, and a PENDING intent. A recoverable object-store step then promotes the staged object, and a second transaction records AVAILABLE/CONSUMED.

Beneath the accounting sits a task-scoped authorization substrate: a control plane that turns an approved task into a signed, budgeted, auditable database session with safe views, row scope, denied fields, operation limits, per-session counters, and receipts. Everything that makes the budget a set rather than a counter is this article's contribution. It is organized around three difficulties: deriving an admitted query's fact sets exactly (1–2); deciding which rewrites preserve identity and which create a new Outcome (3); and keeping retrievable results tied to settled facts under concurrency, failure, replay, and delegation (3–5). The six implementation-backed contributions are:

(1) A typed FactID semantics for accounted Result, positive-output Dependency, and canonical-atom/composite Outcome sets in a restricted algebra. (2) A same-snapshot ordinal companion for a closed reporting fragment. The fragment covers one-Product and connected equijoin queries (two- and three-source joins evaluated end to end; up to 16 roles admitted by the compiler), two-branch union, grouping with HAVING and the admitted aggregates (COUNT, COUNT DISTINCT, SUM, MIN, MAX, exact AVG), and single-Product exact $\{+, -, \times\}$ projections settled as derived cells; bounded nested Views flatten into it. It has no scalar functions, division, OR, subqueries, or outer joins (Section 10 measures the consequence). (3) Profile-relative novelty and atomic three-budget settlement per root family and publication epoch. Novelty is exact by the declared rules and verified as consistent with our reference implementation; it is refined by immutable ordinal dictionaries, compressed bitmaps, and an exact Merkle-radix Outcome set. Settlement precedes `AVAILABLE`, semantic replay has zero novelty, and a rejection leaks a threshold bit bounded by the request budget. (4) Signed, monotonic delegation whose descendants share one root-family ledger, so fan-out cannot multiply release capacity, while per-task request budgets multiply refusal capacity by the approved tree size. (5) Algebraic propositions, a normal-form soundness theorem, an abstract settlement-safety property sanity-checked by two-fact TLA+ models, and executable Go/PostgreSQL differential, generated-plan, and fault-injection evidence. (6) The same machinery for exact retrieval over published document collections: a chunk's visible cells are Facts, exact nearest-neighbour and keyword retrieval are prepared operations behind the same sealed boundary, and an eligibility envelope precedes approval; a formal evaluation finds gateway hit sets equal to direct retrieval and extraction bounded by the budget (Section 6).

2 Context and Related Work

2.1 Closest System Families

The supplement tabulates each family's object, horizon, subject, and enforcement/release coupling.

History, pricing, and provenance. Auditing reconstructs access or determines whether a history fixes forbidden data; simulatable auditing also protects denial [11]–[13]. Query pricing studies determinacy, history, and information gain [3]–[5], and DP accounting and answer reuse bound ϵ [14], [15]. DProvDB [16] is the closest shape—a persistent per-analyst cumulative budget—but meters DP loss over noised views, not an exact declared-identity set (reproduced side-by-side: supplement). Statistical-database inference control studied trackers, query-set overlap, and cumulative auditing [17]–[19]; FactGate's Dependency ledger is a provenance-derived engineering of that lineage: it bounds settled predicates, not inference, and its budget is a set of declared facts where PINQ's is ϵ [20]. Zero-novelty replay is the syntactic analogue of history-aware pricing, and Theorem 1 covers rules R1–R6 only, weaker than arbitrage-freedom. Why/where/how provenance [6], [7], [21], [22] and the GProM/ProvSQL capture mechanisms [23]–[25] explain derivations; FactGate adds no provenance algebra but settles cumulative novelty of typed effects before release.

Data-flow policy enforcement. FlowGuard and DFC/Passant are the closest comparators [1], [2]. Passant enforces optimizer-invariant aggregate predicates over each output tuple's provenance monomials (flow shapes FactGate does not subsume); trajectory policy is its future work. FactGate differs in temporal unit: DFC asks whether a flow is allowed, FactGate whether another allowed flow fits the cumulative contract across queries, retries, pages, rewrites, and descendants (composition not evaluated; Passant artifact replay: supplement).

Authorization, delegation, and agent controls. Task-based and purpose-based authorization scope rights to an activity or a declared purpose [26], [27], and attenuation-only delegation is the macaroon model [28]. Query-rewriting fine-grained control [29] is the per-query comparator for the closed fragment; usage/history control, RLS/VPD, ABAC, and query-policy systems establish ongoing authority, quotas, and rows [30]–[35]. MCP authorization and delegation scope capability invocation [36], [37]. AgentDojo evaluates model behavior under prompt injection [38] and SafeNLIDB inference risk on the model side [39]. Provenance-based access control [40] and CaMeL's tool-layer capabilities [41] decide admission, not cumulative release, and GAAP's persistent cross-tool flow state [42] spans steps and services outside FactGate's scope. FactGate assumes the authorization boundary, settles every descendant against one root ledger, treats the agent and its SQL as untrusted, and claims neither result sanitization, flow tracking, nor inference safety.

Agent retrieval and cumulative accounting. Permission-aware retrieval for enterprise copilots filters each request by the subject's document permissions [43], tool-boundary enforcement narrows each call and leaves aggregate policies out of scope [44], and individually authorized retrievals suffice to reconstruct a corpus [45], [46]. OCELOT [47] budgets an adversary's belief improvement along one trajectory in calibrated min-entropy units, charging each release by operator, field, and sink class rather than by the ledger's prior content; FactGate budgets distinct declared identities under a human-approved contract, charges repeated and split releases once, and settles without any model judgment.

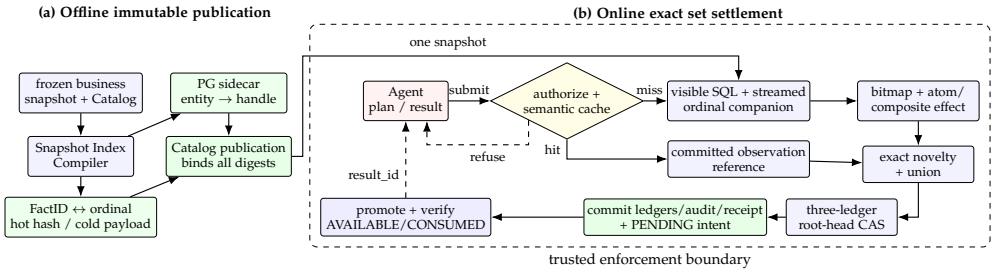


Fig. 1. (a) Offline publication fixes ordinals and the Merkle-radix Outcome set; (b) online exact settlement, promotion, and replay.

3 Exposure Model

Adversary and goal. The agent, its plans, retries, pages, rewrites, and delegated calls are untrusted and may be adaptive. The security goal is bounded governed release: within one approved root family on one publication epoch, no more than B_X distinct FactIDs of each kind become `AVAILABLE`, whatever sequence of admitted queries the agent issues. Four extraction patterns motivate the three budgets, three of them exercised by the evaluation. Page-by-page sweeps are bounded by B_R (Section 5.2); aggregates that read far more base facts than a purpose warrants by B_D (the adaptive trace); delegation fan-out by the shared root ledger (the delegated child's refusal). Probe ladders are bounded by B_O in count but not in recovery: under the default $B_O = 128$ a binary search recovers a 64-bit cell in 64 settled probes, and B_Q adds 128 refusal bits per task. An executed adversary study realizes the bound live (retained pilot, supplement): recovery is capped at $\lfloor B_O/2 \rfloor$ bits under owner-derived tiers (6/4 bits), exact recovery occurs only under a loosened grant, and greedy extraction stops exactly at B_D. Four boundaries hold at this goal: it counts declared identities, not inference; page order is order-only in the Catalog tie-break keys alone (the `ORDER BY` rule below); fresh per-root, per-epoch capacity bounds cross-day release by the approval workflow, not by FactGate; and the settlement decision is two data-dependent counter channels. The first is rejection: a rejected attempt is a chosen membership question over unaccounted facts answered by one bit. It is charged against B_Q only and never B_O, so rejection capacity is $\sum B_Q$ over the approved task tree, not B_X (Section 4): under the default route (B_O=B_Q=128) a k-task tree can settle 128 distinct predicates in total but take $128k$ refusals. The second is the settled response, which returns the per-dimension charge and the ledger's used and limit counters (`exposure`, `exposure_budget`). Each of at most B_O successful novel queries thus reveals $|\Delta_X|$, up to $3\lceil \log_2 B_X \rceil$ bits of query-set-overlap information across the three dimensions. A third, timing channel exists because execution precedes settlement, so a refused query's latency can depend on the footprint it did not release. Execution costs 2.0–0.7 μs per Dependency Fact in the sub-phase pilot; retained refusals with full footprints of 2–18 Facts neither establish nor exclude the dependence against 2.49 ms noise, so the channel is unquantified and outside the guarantee (supplement). Proposition 6 bounds the response payload only. Returning only the remaining budget would be a planning convenience, not a smaller channel (remaining equals prior remaining minus charge). The model trusts identity, attribute, and approval assertions; Catalog and Exposure-Profile authors and the immutable publication pipeline; PostgreSQL execution and snapshot semantics; enforcement and Control services; key management; and object-store access control.

3.1 Publication Epochs and Task Binding

FactGate's upstream boundary is a completed reporting synchronization, not a mutable transactional database.

Definition 1 (Publication epoch)**.** *A publication epoch P_e binds a datasource, its schema, the snapshot bundle mapping every stable role to its immutable reporting publication, the Catalog, and the ordinal dictionaries (descriptor and digest h_e in the supplement). The signed Catalog SHA transitively commits them. The upstream synchronization cutoff is a trusted ETL premise, and a new synchronization creates P_{e+1} rather than mutating P_e.*

A deployment alias such as `latest publication` resolves to a concrete Catalog digest at approval time, never at query time (a draft that must follow a rollover is reissued). Let $\mathsf{Q}(T)$ contain every query issued by an approved root T or any delegated descendant. Then $\mathrm{publication}(T) = P_e$ and $\mathrm{publication}(q) = P_e$ for every $q \in \mathsf{Q}(T)$. The Catalog transitively binds the grant and FactID environment to that epoch. Ledger state, replay metadata, evidence, audits, and receipts carry the same bindings; execution, settlement, and replay fail closed if they differ, so an active root cannot switch publications.

3.2 Facts and Task State

Definition 2 (Approved task)**.** *An approved root task is the immutable authorization boundary $T = (P, S, B, C)$ with ceilings $B = (B_R, B_D, B_O)$. Here P is the signed policy grant over the principal, data products, fields, and scopes, and S is the immutable publication/snapshot binding of the epoch above. B gives the release, positive-output dependency-footprint, and query-outcome ceilings together with the request budget B_Q, the maximum number of settled or rejected attempts; a rejection consumes one unit plus its admission-reserved rows and database time and charges no exposure set. Finally, C is the signed set of semantic and execution constraints and row/time guards. In particular, C binds the effect profile and closed QueryPlan language, type/collation versions, semantic-View contracts, delegation rules, replay identity, and result handling. The monotone accounted ledger $K(T)$ is mutable protocol state and is deliberately* not *a component of the approved tuple.*

The release/dependency algebra treats a FactID as a semantic object (its SHA-256 is only a database index); its semantic environment is $\Pi = \langle pg, \mathcal{T}, UTC, \mathcal{C}, D, h_C \rangle$: a PostgreSQL major version, admitted type domain, timezone, exact deterministic collation name/version map, immutable snapshot map, and catalog/lineage digest. A judgment is defined only for an *admissible* Π: source schemas and entity keys are attested, every collatable field has a deterministic collation, and volatile expressions are absent. Let ϖ be the versioned exposure-profile identifier (Section 3.3), ν a catalog-owned source namespace (not a view alias), s an immutable snapshot, e a canonical stable entity key, a a role-qualified field ID, $\tau \in \mathcal{T}$, and $C_\tau(v)$ its typed canonical value. Base identities are $f_{row} = \langle \varpi, \nu, s, e \rangle$ and $f_{cell} = \langle \varpi, \nu, s, e, a, \tau, C_\tau(v) \rangle$. A derived identity is $f_d = \langle \varpi, \mathcal{B}, k_o, N_\varpi(x), \tau, C_\tau(v), CW(W) \rangle$. Here $\mathcal{B} = \mathrm{sort}\{(\nu, s)\}$ is the relation's snapshot bundle, k_o is the canonical output-row entity key, $N_\varpi(x)$ erases aliases and normalizes the supported expression, and $W : \mathcal{F}_{base} \rightharpoonup \mathbb{N}^+$ is a finite-support witness multiset (empty support allowed). The product identity is the namespace projection of $\mathcal{B}$, not a caller-supplied name. $CW(W)$ length-prefixes the sorted $(\mathrm{FactHash}(f), W(f))$ pairs. The durable key is $\mathrm{SHA256}(d_R \| \mathrm{Canon}(f))$ under the release/dependency separator d_R; Control PostgreSQL stores $\mathrm{Canon}(f)$ and aborts on a same-hash, different-payload row. Outcome facts use disjoint tagged kinds under separator d_O.

Let $N(q)$ be the accepted query normal form, h_N its SHA-256, m the visible row count, h_y the canonical result-observation commitment, and h_c the predicate-context commitment to the approved Catalog, immutable publication/lineage, stable roles, and resolved expressions. Atomization normalizes each accepted literal occurrence by PostgreSQL type, collation name/version, operator, and canonical literal value. After exact deduplication, each semantic atom is $a_i = \langle k_a, \varpi, h_c, p, r, f, e, \tau, \kappa, op, v \rangle$ with $k_a =$ `PREDICATE_ATOM`, p, r, f, e the semantic Product, stable role, public FieldID, and resolved-expression digest, and τ, κ, op, v the type, collation, operator, and canonical literal; $A(q)$ is the atom set $\mathrm{Atomize}(q)$ yields. For any outcome Fact f, its physical key is $\mathrm{FactHash}_O(f) = H(d_O \| \mathrm{Canon}(f))$. Let h_A commit the sorted $\mathrm{FactHash}_O$ values of $A(q)$ and $k_c =$ `COMPOSITE_OUTCOME`. FactGate creates one semantic composite object $c_q = \langle k_c, \varpi, \mathrm{profile}(N), h_N, h_y, m, h_c, h_A, |A(q)| \rangle$; the query's Outcome Facts are $\mathcal{F}_O(q) = A(q) \cup \{c_q\}$ and $M_O(q)$ is the set of their $\mathrm{FactHash}_O$ values. Repeated literals and supported rewrites thus reuse atom identities. The composite commits to the normalized query, exact result observation, and complete atom set, so every distinct successful query (every page included) adds one composite, and B_O also caps distinct successful queries. The Merkle-radix set stores $M_O(q)$; empty results and scalar zero stay explicit, and the signed grant bounds literal occurrences, unique atoms, and payloads before settlement.

Proposition 1 (Fact identity well-definedness)**.** *For every value in the supported PostgreSQL type domain, canonical encoding is deterministic and injective over the typed semantic payload (that is, over PostgreSQL semantic-value equivalence, not raw float NaN bit patterns; proof in the supplement).*

Definition 3 (Exposure effect)**.** *For query q over snapshot D_s, the exposure effect $\mathcal{E}(q, D_s) = \langle \mathcal{F}_R, \mathcal{F}_D, \mathcal{F}_O \rangle$ contains accounted result facts (retaining the wire label* `release`*) $\mathcal{F}_R$ and positive-output dependency-footprint facts $\mathcal{F}_D$ under a versioned profile ϖ. The latter are the base facts the inductive rules below select for successfully derived output rows and cells; the admitted language requires every* `ORDER BY` *column to be selected, so only the Catalog tie-break suffix of Page is order-only and outside this profile. $\mathcal{F}_O = A(q) \cup \{c_q\}$ contains the predicate footprint and composite query/result identity defined above.*

Example (charged by the operator rules of Section 3.5). Over a five-row Product with fields $(id, dept, amt)$, the query $\pi_{id,amt}\sigma_{dept='S'}$ selects rows r_1–r_3. It charges 6 Result cells, 12 Dependency facts (each selected row's row fact, its $dept$ cell read by the predicate, and its two visible cells), and 2 Outcome facts (the atom $dept$='S' and the composite). Splitting it into π_{id} and π_{amt} charges $0/0/2$ (two new composites, the same atom); a page returning r_2, r_3 charges $0/0/1$; adding $amt > 5$ charges one new atom and one composite and no new base facts; an exact replay by the root or by a delegated child charges $0/0/0$.

For a successful q, settlement commits the complete candidate Result set before the later, recoverable publication, so $F_R^{\mathrm{available}}(q) \subseteq F_R^{\mathrm{accounted}}(q) = F_R^{\mathrm{candidate}}(q)$; download is untracked, so accounting only upper-bounds observation.

Definition 4 (Task-scoped cumulative exposure)**.** *Let $Q = \langle q_1, \ldots, q_n \rangle$ be a finite query-attempt history for a root and its descendants, and let $Q^+ \subseteq Q$ contain exactly the queries that successfully settle under T. For $X \in \{R, D, O\}$, $\mathcal{F}_X(q, S)$ abbreviates $\mathcal{F}_X(q, D_s)$ for the snapshot D_s and profile that S and C transitively bind; the two spellings denote one set. The task exposure is*

$$\begin{aligned} E(T, Q) &= \langle E_R(T, Q), E_D(T, Q), E_O(T, Q) \rangle, \\ E_R(T, Q) &= \bigcup_{q \in Q^+} \mathcal{F}_R(q, S), \\ E_D(T, Q) &= \bigcup_{q \in Q^+} \mathcal{F}_D(q, S), \\ E_O(T, Q) &= \bigcup_{q \in Q^+} \mathcal{F}_O(q, S). \end{aligned} \tag{1}$$

The components are accounted result exposure (wire label `release`*), positive-output dependency footprint, and query-outcome exposure. Failed or rejected attempts contribute no set because FactGate commits none of their effects; the rejection bit is not accounted here. We write the singleton history as $E(T, q)$.*

For every root task, the ledger service separately stores previously accounted sets $K(T) = \mathcal{K}_T = \langle \mathcal{K}_T^R, \mathcal{K}_T^D, \mathcal{K}_T^O \rangle$ and the approved limits $B = \langle B_R, B_D, B_O \rangle$. Settlement computes

$$\Delta_T^R(q) = \mathcal{F}_R(q, D_s) \setminus \mathcal{K}_T^R, \tag{2}$$

$$\Delta_T^D(q) = \mathcal{F}_D(q, D_s) \setminus \mathcal{K}_T^D, \tag{3}$$

$$\Delta_T^O(q) = \mathcal{F}_O(q, D_s) \setminus \mathcal{K}_T^O. \tag{4}$$

FactGate measures cardinality independently in each dimension. A query may settle only if $|\mathcal{K}_T^R| + |\Delta_T^R| \le B_R$ and $|\mathcal{K}_T^D| + |\Delta_T^D| \le B_D$ and $|\mathcal{K}_T^O| + |\Delta_T^O| \le B_O$.

3.3 Exposure Profiles as Clearance Manifests

Definition 5 (Profile-relative clearance exactness)**.** *For an approved $T = (P, S, B, C)$, C binds one versioned Exposure Profile ϖ. For any admissible Π and successful query in the closed language, write $\mathcal{F}_X^{\varpi}(q)$ for the existing component $\mathcal{F}_X(q, S)$ with its profile binding made explicit. FactGate exactly computes the canonical Result, Dependency, and Outcome sets $\mathcal{F}_R^{\varpi}(q), \mathcal{F}_D^{\varpi}(q), \mathcal{F}_O^{\varpi}(q)$ and their componentwise novelty*

$$\Delta_T^X(q) = \mathcal{F}_X^{\varpi}(q) \setminus \mathcal{K}_T^X, \qquad X \in \{R, D, O\}.$$

This definition fixes the scope of the word exact*: governance accounting exactness under the signed contract and closed semantics, asserting nothing beyond the results that follow, and established empirically only as consistency with our reference implementations (Section 9). FactIDs are accounting identities, not estimates of information content, knowledge, or privacy loss; FactGate prevents unaccounted governed release under a task-bound, versioned profile.*

Example (cont.). Under budgets $B_R/B_D/B_O = 6/12/7$ the five-row episode of Section 3 ends with ledger totals $(6, 12, 7)$; a next query $\sigma_{dept='T'}$ needs new base and Outcome facts, so its Δ_T no longer fits, and it is refused uncharged—the ledger still reads $(6, 12, 7)$. Exactness rests on four accounting obligations, proved for the closed language in the supplement (O1–O4). (O1) Every selected cell of a successful result has its materialized identity in $\mathcal{F}_R$. (O2) $\mathcal{F}_D^{\varpi}$ is exactly the declared inductive footprint (retained predicate fields, join keys, group keys, complete aggregate arguments), excluding rejected rows and non-output pages. (O3) The listed rewrites, splits, page partitions, overlaps, retries, join support, and delegation cannot reduce cumulative charge. (O4) Each success creates one composite and its atoms, exact replay reuses them, and a rejected attempt creates none, charged only to B_Q.

Six obligations separate the assurance layers (O1–O4 semantic claims, O5 models plus transaction tests, O6 binding); none alone proves the implementation.

Property 1 (O5: No unaccounted governed release)**.** *Let* $\mathsf{AVAILABLE}(q)$ *mean that q's canonical artifact has reached the* `AVAILABLE` *state. For every modeled query and $X \in \{R, D, O\}$,*

$$\mathsf{AVAILABLE}(q) \Longrightarrow \mathcal{F}_X^{\varpi}(q) \subseteq \mathcal{K}_T^X \ \wedge \ |\mathcal{K}_T^X| \le B_X.$$

It is the release-side counterpart of consumable credentials and quotas [48], [49]. Execution and private staging are not release; settlement may leave an artifact `PENDING`, so the converse fails and rejection cannot create `AVAILABLE`. Evidence: bounded artifact/ledger models plus Control-PG fault-injection, receipt, and availability tests, not a mechanized proof.

O6: Profile binding. C binds effect/normal-form profiles, type/collation, View contract, replay identity, and result handling, and every stage binds one task/Catalog/publication/dictionary context, so a changed context cannot reuse an old replay.

Three-way reconciliation. Effect manifest, root-ledger heads, and artifact/receipt/availability evidence must identify one settled observation and intent; any mismatch fails closed.

Proposition 2 (Outcome separation and replay)**.** *For successful queries in one predicate context, exact replay and supported equivalent rewrites produce the same atom set and composite identity and hence zero incremental Outcome charge. Changing one canonical predicate component changes that atom except with a SHA-256 collision; changing the complete normal form or result observation changes the composite except with a collision. Consequently related probes reuse unchanged atoms but charge each novel atom and novel composite exactly once, including for zero/empty answers (proof in the supplement).*

3.4 Closed Typed Query Language

Operational multi-source SQL first lowers to $G_J = \langle V_J, E_J, \Theta_J \rangle$: 2–16 distinct Catalog stable-role nodes and connected equality edges. Stable-role ordering and a deterministic fold erase aliases and accepted parenthesization, so the same connected graph has one core term. Repeated roles, self edges, duplicate equalities, and type or collation disagreement reject; the operational bounds and construction are supplemental.

The complete core syntax is

$$\begin{aligned} e ::= {} & \mathrm{Scan}[\nu, s, r, \Gamma] \mid \sigma_{p;F}(e) \mid \pi_A(e) \mid \pi_{A;x}(e) \mid e \bowtie_{\Theta} e \\ & \mid e \cup_{\delta} e \mid \gamma_{G;\mathcal{A}}(e) \mid \mathrm{Page}_{O,K,o,l}(e), \\ p ::= {} & b \mid p \wedge b, \qquad b ::= a \ op \ c, \\ op \in {} & \{=, \neq, <, \le, >, \ge\} \cup \{\texttt{LIKE}, \texttt{IN}, \texttt{NOT IN}\}, \end{aligned}$$

where F is exactly the fields read by p, Θ is a nonempty conjunction of equalities, $\cup_{\delta}$ is union-distinct, $\mathcal{A} \subseteq \{\texttt{COUNT(*)}, \texttt{COUNT}, \texttt{COUNT DISTINCT}, \texttt{SUM}, \texttt{MIN}, \texttt{MAX}, \texttt{AVG}\}$ (`AVG` exact-numeric only), $\pi_{A;x}$ adds one single-Product exact $\{+, -, \times\}$ derived cell x, and `HAVING` is σ over γ's output (rules in the supplement). No implicit SQL construct is hidden in e; the excluded constructs and the typing judgment $\Pi \vdash e : \Gamma$ are listed in the supplement.

The View front end lowers a bounded acyclic DAG of projections, inner equijoins, literal conjunctions, and at most one grouping barrier into this core, rejecting richer SQL or unsafe dependencies; no general View equivalence is claimed.

Predicates use PostgreSQL three-valued logic and retain only true rows, whereas grouping/distinct use typed `IS NOT DISTINCT FROM`. The closed canonical domain fixes integer/float widths, exact numeric and temporal encodings, deterministic collation name/version, `jsonb` structural equality, and PostgreSQL aggregate result types. Non-associative floating SUM, unsupported casts, and overflow are undefined and settle nothing; the complete type domain and encoders are supplemental.

3.5 Annotated-Bag Evaluation and Effect Extraction

A relation is a typed PostgreSQL bag with unique occurrence keys and optional proven total order; rows and cells carry support S (lineage [21], [50]), a positive witness multiset W (how-provenance [6]), canonical lineage, and optional materialized release identity, and every operator preserves

$$\mathrm{support}(W_r) = S_r, \qquad \mathrm{support}(W_a) = S_a. \tag{5}$$

Same-proof composition unions support and adds multiplicities; alternative proofs union support and take pointwise maximum, so alternatives are idempotent while join

fanout remains visible. Row identity $\mathrm{RID}(R,t)$ hashes the canonical schema, snapshot bundle, and complete operator-specific key material under a typed, domain-separated H (construction and collision checks are supplemental).

Evaluation is the partial judgment $\Pi; D \vdash e \Downarrow R$: a failed premise yields no relation, effect, settlement, or release; every rule constructs each output cell explicitly (complete rules, NULL/empty-input cases, and page bounds are in the supplement). Union is bag-aware and idempotent only when its typed tuples are proven duplicate-free, grouping/distinct equivalence is `IS NOT DISTINCT FROM`, floating-point `SUM` is rejected because the bag fixes no fold order, and pagination appends Catalog keys until the order is total; the relation is a partial function, not a semantics for arbitrary SQL. Materialization $\mathrm{mat}(R,t,a)$ reuses a cell's pre-existing release fact when value, type, bundle, and support or expression/witness still validate, and otherwise mints the derived identity; the effect is

$$\mathcal{F}_R(R,V) = \{\mathrm{mat}(R,t,a) \mid t \in R,\ a \in V\}, \tag{6}$$

$$\mathcal{F}_D(R,V) = \bigcup_{t\in R} S_r(t) \ \cup \bigcup_{t\in R, a\in V} S_a(t), \tag{7}$$

$$\mathcal{E}^{ri}_{\Pi}(e,D,V) = \mathrm{normalize}\langle \mathcal{F}_R(R,V), \mathcal{F}_D(R,V)\rangle, \tag{8}$$

where $S_r(t)$ is row t's support (the base row facts and the predicate, join-key, group-key, and aggregate-argument cells its derivation read) and $S_a(t)$ the support of its cell a. Query bags and witness multiplicity are retained during composition, while the two base ledgers are sets. Equation (8) is the declared positive-output dependency-footprint profile: it is exact with respect to these inductive rules, not with respect to arbitrary causal provenance or physical source access. Empty answers have an empty base effect except the single visible row of a global aggregate; FactGate still adds the tested predicate atoms and one composite outcome.

Proposition 3 (Representation closure and dependency determinism)**.** *For every admissible* Π*, typed finite* e*, and immutable* D*, evaluation has at most one result, satisfying Eq.* (5) *with canonical schema/bundle and unique row keys, valid as input to every well-typed parent rule; fixed* Π*,* e*,* D*, and visible set* V *uniquely determine* $\mathcal{F}_D$ *and* $\mathcal{E}^{ri}_{\Pi}(e,D,V)$ *whenever evaluation succeeds.*

Proofs in the supplement.

3.6 Conservation Properties

The properties below cover only the stated operators and rewrites, not arbitrary equivalent SQL; the supplement proves Fact-set normalization and the restricted selection–projection invariance the rewrite tests use.

Definition 6 (Profile-indexed typed normal form $NF_{\Pi,\varpi}$)**.** $NF_{\Pi,\varpi}$ *maps accepted, well-typed inputs to canonical terms under the normal-form profile the contract binds (rules R1–R6 below); equality is syntactic. Its typed digest is the query-semantic component of replay/outcome identity, distinct from final-ledger Fact-set* normalize*.*

Theorem 1 (Typed normal-form soundness)**.** *For admissible* Π*, fixed profile* ϖ*, immutable snapshot map* D*,* q_1, q_2 *in the closed algebra or accepted graph-lowerable SQL fragment, and the same visible field set* V*,*

$$NF_{\Pi,\varpi}(q_1) = NF_{\Pi,\varpi}(q_2) \Longrightarrow \mathcal{E}_{\Pi}(q_1,D,V) = \mathcal{E}_{\Pi}(q_2,D,V).$$

This is not completeness for arbitrary equivalent SQL: normalization neither invents nor removes transitive/redundant equalities.

The theorem is the analogue, for the release/dependency effect and rules R1–R6, of the invariance of semiring provenance under equivalent positive-algebra rewrites [6], [7]; what is new is the effect function it preserves, not the proof pattern. The rewrite set is exactly six rules. (R1) Erase aliases and display names to Catalog stable roles. (R2) Canonicalize qualified field, function, and operator spellings and typed literals. (R3) Sort commutative components—AND conjuncts, projections, aggregates, group keys, union operands, and join operand/predicate order—by typed stable identity. (R4) Fold an accepted connected equi-join graph into one canonical binary term (after the graph boundary only, never reassociating arbitrary SQL). (R5) Collapse an identical duplicate union branch only when its child carries a full-tuple distinctness proof (Union and Group produce one, Select and Page preserve it, Project preserves it only for an unchanged schema, Scan has none). (R6) Canonicalize the Catalog page-order suffix. The supplement states a per-rule effect-preservation lemma with these side conditions; RQ2's zero-charge guarantee covers exactly R1–R6.

For a fixed approved task, let ϖ_T be the normal-form profile bound in C and define the canonical replay descriptor

$$\mathrm{RK}_T(q) = \langle P, S, C, \varpi_T, NF_{\Pi,\varpi_T}(q), \mathrm{params}_{\Pi}(q), \mathrm{encver}\rangle, \tag{9}$$

where every component uses the typed, length-delimited encoding bound by the Grant and Catalog; the implementation indexes a digest of this descriptor but verifies the bound metadata before accepting a hit. Descriptor equality fixes policy, publication, effect/View contracts, typed parameters, and normal form, hence all three effect sets, so exact replay has zero novelty; the converse is not claimed outside the stated rules. A semantic-View root compiles to a closed core term with a binding β_v; DAG expansion retaining β_v preserves the visible bag and all three effect sets (proof in the supplement); a direct terminal query can carry different Outcome FactIDs.

Proposition 4 (Base-ledger split/page/join conservation)**.** *Merging release/dependency observations for disjoint projections or a complete page partition equals observing the combined result. Overlapping pages and repeated terminal observations charge each release/dependency FactID at most once; join fanout changes witness multiplicity but not the union of dependency support; predicate atoms are reused exactly when their canonical footprint components agree (proof in the supplement; the support/multiplicity split is the lineage projection of semiring provenance [6], [7]).*

4 Online Accounting

Terms: a publication's *hot artifact* holds its entity-key-to-ordinal map and verified hashes, its *cold* payloads the content-addressed Fact bytes, and its *sidecar* the PostgreSQL table mapping attested entity keys to ordinals; the *root head*

is the Control row holding a family's three set digests and epoch, advanced only by compare-and-swap (*CAS*).

Formal semantics and the RQ1 oracle cover scan/select/project, group (with HAVING), inner equijoin, union-distinct, and page. Online lowering admits one product, a connected 2–16-source equi-join graph, or two-branch union-distinct, optionally grouped; the SQL-AST path covers the first two and the semantic-View path eliminates a bounded nested DAG into them. Union is structured-plan only, pagination is one-product only, and "online" does not mean arbitrary SQL. For joins, the PostgreSQL AST and Catalog types feed the canonical G_J builder, which emits the multi-way join operator and the deterministic binary SQL/effect fold. Unsupported joins, set trees, and pagination fail closed, and callers cannot supply roles, SQL fragments, FactIDs, or provenance. For semantic Views, one catalog snapshot checks definitions, interfaces, and dependency edges, expands safe acyclic Views to materialized leaves, and emits the same core. Four digests (root definition, dependency closure, canonical plan, public interface) bind into the Grant and query; drift requires a new approval, and execution resolves least-privilege terminal scopes before reservation.

4.1 Snapshot Publication and Ordinal Compilation

The publication compiler turns each frozen snapshot into immutable per-row/per-field segments, sorting full Fact hashes and assigning 32-bit ordinals (a FactID–ordinal bijection D_j per segment; Roaring compression changes representation, not membership [51]); a sidecar maps entity keys to ordinals, and one manifest binds snapshot, schemas, sidecar, dictionaries, and digests. Online, visible SQL and the companion run in one read-only `REPEATABLE READ` transaction. The companion retains every required dependency, and a k-way hash merge reproduces the witness commitment. Release/Dependency use segmented Roaring bitmaps, and Outcome hashes enter an exact Merkle-radix set. Invalid handles, manifests, encodings, overflow, or truncation fail closed.

Decoding is exact: within one segment, bitmap OR, ANDNOT, and popcount are set union, difference, and cardinality, and the streamed bitmaps decode to exactly the release and dependency components of $\mathcal{E}(q, D_s)$ (proof in the supplement). For Outcome, hash prefixes route content-addressed leaves and blocks, but membership compares the full SHA-256 value.

Proposition 5 (Exact incremental Outcome-set refinement)**.** *Let K_O be a valid Outcome root and C_O a candidate. Loading every leaf family whose 16-bit prefix occurs in C_O, performing exact full-hash difference/union there, rebuilding only their blocks, and reusing all other references yields* $\mathrm{Decode}(K'_O) = \mathrm{Decode}(K_O) \cup C_O \setminus \mathrm{Decode}(K_O)$ *exactly. Any changed member, count, prefix, chunk, block, or root changes a verified digest barring SHA-256 collision.*

Argument in the supplement (prefix routing partitions the full-hash universe; comparison is over all 256 bits; digests form a Merkle tree [52] over radix blocks).

4.2 Atomic Ledger Settlement and Fail-Closed Artifact Publication

For one successful query (Fig. 1b) the enforcement layer reauthorizes, reserves resources, and looks up semantic replay under the signed grant/publication binding. On a miss it executes the visible and ordinal statements in one Business-PG snapshot, derives all three effects, and stages encrypted Parquet (a hit loads a committed observation; staging is never AVAILABLE). One Control-PG transaction then computes exact bitmap/full-hash novelty against the root head, rejects any B_X excess, CASes all three manifests (conflicts reread and recompute every dimension), and settles resources, observation, PENDING intent, audit, and the signed receipt. A second transaction co-commits AVAILABLE and `QUERY_RESULT_CONSUMED` after verifying promotion to the canonical object, then returns the ID.

Only root heads are mutable: pre-settlement failure rolls everything back, while post-settlement promotion failure neither refunds nor re-executes but resumes the charged PENDING artifact idempotently (supplement).

Accounted means settlement committed, *available* canonical promotion plus consumption audit, and *observed* an untracked download, so $\mathsf{Observed} \subseteq \mathsf{Available} \subseteq \mathsf{Accounted}$ and $\mathsf{Available} \Rightarrow \mathsf{Settled}$, not conversely.

Semantic replay binds task, grant, typed normal form and parameters, Catalog, publication, dictionaries, and every profile and encoding version; a hit skips Business SQL but still reauthorizes, meters, audits, receipts, and re-encrypts, and P_{e+1} roots cannot hit P_e entries.

4.3 Receipts and Failure Semantics

A signed Ed25519 settlement receipt binds accounting evidence, result and object hashes, the `PENDING` intent, and registration audit without exposing values or FactIDs; failure cases and receipt binding are detailed in the supplement. A rejected attempt returns only the code `EXPOSURE_BUDGET_EXHAUSTED`—no dimension, count, or row (evaluation tables name the oracle's projection, not a response field). It charges no exposure, in particular no Outcome atom, and is recorded as a failed query (contributing no set, Definition 4) that consumes B_Q and its admission reservation (Section 3). B_Q is an exposure parameter, not a resource guard.

Proposition 6 (Probing bound)**.** *Each rejected attempt reveals one comparison outcome—whether its novel effect fits the remaining budgets—and consumes one unit of the attempting task's own B_Q, which the prototype accounts per task. A delegated child signs its own B_Q no larger than its parent's, so a task tree of k tasks obtains at most $\sum_{i \le k} B_Q^{(i)}$ comparisons in total, independently of B_R, B_D, and B_O. Each successful novel query additionally returns its charge $|\Delta_X|$, the size of its footprint outside the ledger; at most B_O such counts exist per root family.*

That a refusal itself answers a query is the classical auditing observation [13], [18], [19]. The bound is on the response payload: execution and derivation precede settlement (Section 4), so a refused query's latency may be a function of the footprint that was not released, a timing channel the model omits (Section 10). The bound counts comparisons, not what a comparison reveals about unreleased facts; the

protocol is not a simulatable auditor and makes no denial-noninterference claim (supplement).

5 DELEGATION

5.1 Root-Family Conservation

Every child grant is a human-approved authorization (the prototype's only approval mode is manual), so autonomous delegation is not supported and conservation is evaluated for human-approved trees. A child authorization signs both `root_task_id` and `parent_task_id`. Its human subject, catalog, datasource, schema, and profile remain fixed, while products, columns, scopes, sensitivity, expiry, and budgets may only narrow; every execution revalidates the signed parent grant and ancestor chain; View-bound descendants inherit the parent's contracts and cannot add or replace one.

All descendants resolve to one root ledger: if a parent and child request the same fact, only the first successful settlement accounts it. Revoking the root blocks new descendant queries while a terminal request ID stays readable, and creating workers or task nodes cannot create fresh exposure capacity. Every child grant is itself a human-approved authorization on the Catalog's route (the prototype's only mode is manual approval), so the tree's size, and the summed request budget of Proposition 6, is set by approvals, not by the agent. *Example (cont.).* A delegated child re-issuing π_{id} replays at $0/0/0$ against the shared ledger, and $\sigma_{dept='T'}$ is refused by the same Δ_T. Settlement checks the settling task's signed ceiling against the family total, so a lower-ceiling child adds novel facts only while the family total stays within it and otherwise only zero-novelty replays: a narrower child cannot widen the family.

Conservation stops at the root and at the publication epoch: a second root for the same principal, or a root approved on P_{e+1}, has an independent ledger, so two roots on one Product double its capacity and unchanged content may be accounted again under P_{e+1}; an existing root continues against P_e through a retained route or fails closed. Cross-root deduplication would need a tenant-wide identity outside the task-scoped guarantee.

Because Definition 4 sums over the queries that settled, $K_X(T) = E_X(T, Q)$ is not itself a theorem; the claim is a safety property of an abstract settlement system. A root state is (K, e), three decoded sets and an epoch. A settlement transaction for q reads (K, e), computes $\Delta_X = \mathcal{F}_X(q, S) \setminus K_X$, and attempts one compare-and-swap that publishes $(K \cup \mathcal{F}(q, S), e + 1)$ only if it sees epoch e and $|K_X| + |\Delta_X| \leq B_X$ for all X; a stale swap rereads, and a rejected or failed attempt publishes nothing. Root-family transactions, descendants included, interleave arbitrarily, but each swap is atomic.

Property 2 (Atomic root-family budget safety)**.** *In every execution of this system, every committed head equals the componentwise union of the effects of exactly the queries whose swap succeeded and satisfies $|K_X| \leq B_X$ for all $X \in \{R, D, O\}$; the property is unchanged when K_X is represented by dictionary bitmaps or the Merkle-radix set (Proposition 5 and the supplement's bitmap-refinement proposition).*

Argument. Induct on successful swaps in linearization order [53]: the initial head is empty, each successful swap extends the head by exactly the effect it computed against the head it read, and the bound check precedes the swap; the `ExposureLedger` model (Section 8) and the Control-PG race and fault-injection tests are correspondence evidence, not a mechanized refinement.

5.2 Budget Calibration

Catalog-issued capacity. Profile units are distinct accounted identities per dimension, separate from resource guards (B_Q, rows, DB time) and from quantities they do not estimate (knowledge value, privacy loss, minimal causes, bits, or DP ϵ; table in the supplement); unit weights enable exact set union, not harm accounting. These declarations are neither risk scores nor optimization targets: 1,000 permits 1,000 declared units, and the bound profile must remain acceptable when fully used. B_O bounds distinct successful queries (every page settles one composite) plus distinct predicate atoms: unlike the counters Section 1 dismisses it charges a repeated outcome once and bounds no release size, which B_R and B_D do. Which dimension binds depends on the purpose archetype: B_R for detail review, B_O for single-Product aggregate reporting (where B_D degenerates to the Product's fact count), B_D for cross-Product joins; B_Q is a small multiple of B_O throughout, and the accounting is compliance accounting with quantified probe capacity, not extraction resistance (archetypes worked in the supplement). The supplement works the campaign's bounded profile (its budgets and refusal point: Section 9), the count-probe ladder, and a page sweep—a toy department, not a realistic-scale calibration.

6 KNOWLEDGE PRODUCTS

Because the contract, settlement, and delegation are stated over Fact sets, not SQL, a published document collection maps layer by layer onto data product, Fact, prepared operation, and effect, and one gateway governs both. A Knowledge Product is a governed collection with stable document identifiers, a deterministic segmentation function, typed extraction rules, and an attribute vocabulary (department, role, classification, validity); an index publication epoch freezes the segmentation, the embedding model, every chunk's content digest and labels, and the extracted values. Chunks and document versions play rows, and a chunk's visible cells and each extracted typed value play cells. The language is `search` (exact nearest-neighbour retrieval, or the chunks containing every keyword ranked by matched characters; $k \leq k_{\max}$), `fetch` of chunks already returned to the root family, and `aggregate` over an extracted field; each is prepared through the same sealed boundary as `execute_plan` and settles on the same three ledgers. $\mathcal{F}_R$ holds the visible cells of each returned chunk, $\mathcal{F}_D$ adds the rows and filter cells the hit set depended on, and the Outcome composite commits to the canonical atoms, the normalized query, k, and the hit set. The mandatory scope is injected as atoms, and before any approval route an eligibility envelope, a Catalog predicate over the subject's attested attributes, rejects requests outside the subject's own scope; inside it an approver may only narrow, so an erroneous approval stays bounded by B and the TTL. Root-family safety and

delegation conservation carry unchanged; the normal-form theorem does not (supplement portability table).

Evaluation. A formal round (3 fresh deployments per experiment from one frozen commit, all requests over HTTP; supplement) ran five pre-registered experiments on synthetic corpora of 48,183 and 96,401 chunks. Under an 800-cell Release budget all 27 scripted extraction runs stopped at budget exhaustion with at most 7.6% of their department's chunks, where per-query access control without a ledger let a vocabulary scan obtain 63.4% on average. Gateway retrieval matched direct exact retrieval member for member in 100.0% of 600 questions at $k = 10$, as at $k = 5$ and 20, and identically across deployments; keyword search returned the hit list of direct `LIKE` execution, members and order, in 100.0% of 1350 queries at each k, and a sweep over frequent keywords stopped at budget exhaustion with at most 7.4% of the department's chunks, against 43.7% without a ledger. Retrieval quality is reported, not claimed, and missed its pre-registered thresholds (recall@10 79.7% near-verbatim and 55.3% paraphrased, against 80% and 60%); legitimate questions under the extraction budget met no refusal, but only 60.0% (vector) and 53.3% (keyword) found their source document within five hits (pre-registered 90% and 80%); direct retrieval misses the same. Requests outside the eligibility envelope were refused before any approval draft (177 of 177 cross-department and 108 of 108 above-clearance requests), and a misapproved grant leaked at most 200 chunks against its bound of 200. A search at $k = 20$ over 96,401 chunks takes 171.3 ms through the gateway against 14.8 ms direct, a keyword search 64.3 against 0.3 ms (medians). A count over a collection-scoped value field returns 10 distinct values and charges 50 Dependency Facts, where a document-scoped one counts 116 contracts at 580.

7 System Realization

The Go prototype uses separate Control and Business PostgreSQL 16 databases; the Catalog declares Views, scopes, immutable publications, entity keys, operations, approval routes, and profiles, and startup and per-query attestation verify every binding. Each instance is Catalog-bound, so rollover routes old roots to retained P_e instances and new approvals to P_{e+1}. MCP tools cover discovery, approval, execution, delegation, budgets, results, and receipts; direct SQL is lowered to the same canonical plan or fails closed, so no path executes agent-authored text; results use AES-GCM and audits form a hash chain. A text search is embedded inside the trusted boundary by a service that sees query and chunk text but no ledger or grant; the commitment digests the normalized text, and a missing or mismatched embedding model fails the search closed. Governed statements pin `search_path` to `pg_catalog`, so pgvector is installed there. FactGate fronts the governed serving tier and replaces none of the surrounding lake, Catalog, IAM, OA/BPM, object-store, RLS/DFC, or audit systems (supplement interface table; not a production deployment). Crashes, concurrency, stale bindings, replay, and promotion failure are in scope; compromised trusted authorities, direct bypass, colluding independent roots, and external-tool leakage are not.

8 Three-Layer Assurance

Paper theorems establish representation-independent Fact identity, deterministic effects, and exact novelty for the closed algebra, and Property 2 states root-family conservation over an abstract settlement system; none proves the parser or implementation. Four bounded TLC models [54] check abstract root settlement, FactSet/bitmap correspondence, exact Outcome-set settlement with collision fail-closed safety, and PENDING/AVAILABLE recovery at small configurations (two facts per dimension, limits $1/1/2$; 410,766, 122,976, 20, and 1,324 states; bounds in the supplement), sanity checks that abstract away SQL, rows, hashes, signatures, and ciphertext. Executable Go/PostgreSQL differential, concurrency, tamper, and fault-injection tests check implementation correspondence (the supplement maps layers and gaps). Among them, a randomized root-family test (supplement) settles random observation sets over random delegation trees, sequentially and in concurrent bursts, against the live Control store, checking after every step that the committed head equals a plain-set union, respects every ceiling, and matches the admit/refuse decision and exact-novelty charge. We claim bounded model checking plus implementation evidence, not a mechanized refinement.

9 Evaluation

9.1 Questions and Reproducibility

RQ1 asks ground-truth Fact membership; RQ2, rewrite and charge invariance; RQ3, conservation under adaptive execution, delegation, and concurrency; RQ4, ordinal/bitmap acceptance (answered by the campaign's Scale profile; the archived V4 run is reported only in the supplement); and RQ5, daily publication and retained routing. RQ1–RQ3 regenerate digest-bound corpus `5704d22606b0`, and every call uses a complete signed profile. Medians and p95 are Type-7 quantiles [55] over the stated samples; ProvSQL provenance capture is the same-host cost reference (Section 9.2).

RQ1's oracle (24/24 cases) is a separate implementation over 16 rows that imports no production policy code. The campaign's oracle package is likewise a separate implementation of the FactID wire contract, written by us from the same specification, whose import boundary excludes every production package (a dependency test enforces this). It fixes the expected Fact cardinalities of all 2,214 Scale and trace samples and is linked member by member, through the activated publication dictionaries, to every committed Dependency set of the Scale and ProvSQL cells (7,560 and 270 set verifications, none with a missing or extra ordinal, up to 2,070,000 Facts per set). Agreement is thus implementation-to-specification: such oracles cannot detect specification errors or a shared misreading of PostgreSQL semantics, so we claim consistency with our oracle, not independent validation; RQ1's 24 and RQ3's 5 cases are regression evidence, not statistical evidence. One check shares nothing with our specification: a pilot-class rerun (3 samples) expands ProvSQL's own circuit to its base tuples on the paired join cells and finds them equal, member by member, to the oracle's 336,003 base-row facts, to which FactGate's Dependency sets are linked. The archived 1,035,013-Fact RQ4 check (30/30 gates, one index build) shares the

FactID encoder and is only derivation-independent. RQ1–RQ3 and RQ5 are regenerated at the submission commit; the campaign is sealed at that commit (Section 9.3). RQ2 (19 cases, 0 mismatches) requires equal Result/Dependency sets and zero bidirectional charge. RQ3 (5 cases, deterministic and as PostgreSQL race tests) covers split, page, retry, snapshot identity, joins/unions, and delegation. RQ5 passes 5/5 rollover checks across 3 transitions (build and activation timings: supplement). Outcome regressions cover empty probes, replay, radix tamper, and digest binding. Beyond the hand-written cases, a generated-plan campaign (5,000 random plans over 50 fixtures, spanning selections, projections, equijoins, aggregates, and pages) compares the production algebra with a separate reference evaluator: 0 set and 0 hash mismatches over 55,801 Result and 202,306 Dependency Facts, with split and page unions equal to the complete query in all 380 checks (supplement).

The core workloads (the campaign's *Baseline* profile): S1, a two-column projection of the frozen 50,000-order corpus under an order-key threshold (SF1: 5,000 orders; SF10: all); S2, the orders–lineitem equijoin grouped to three rows; S3, a one-column projection over 1,000–45,000 rows of the 414,000-row exposure-scale Product; S4, the depth-4 semantic-View form of S2/SF1; S5, a two-branch union-distinct of S1's shape; S6, result-heavy releases of 100–100,000 rows with 4 or 16 columns. Modes are direct (the same SQL against PostgreSQL without FactGate), novel (the first governed execution), and the replays; "sealed campaign" denotes the frozen protocol's runs (retained pilots are supplementary, never eligible). The frozen campaign (identified in the supplement) executed all 172 frozen cells: 125 profile cells across 33 fresh deployments of 11 profiles, each deployed 3 times, plus 36 Scale and 11 Compiler deployment-free cells. Every one of its 9,393 measured samples passed (all per-sample invariant and oracle checks; deterministic, not statistical), with a verified Receipt on every governed arm. Compiler-admitted join widths and role counts beyond the measured cells stay unmeasured (supplement). Table 1 gives the per-cell medians: across the 58 Baseline cells (sixteen workload/scale pairs with direct, novel, and the protocol-defined replay cells) a novel governed query costs 21.8–144.0 times its direct PostgreSQL equivalent. That cost follows the total Fact effect the query materializes, not its visible row count. S2/SF10 returns 3 grouped rows over 1,200,000 dependencies in 2.46 s, longer than S1/SF10 needs to release 100,000 Facts (0.33 s). The result-heavy S6 cells cost more per Fact than S2, so Release materialization is at least as expensive per Fact as Dependency derivation. Settlement is not where exactness is paid: the Scale profile below settles against a million-Fact history in 163 ms. A sub-phase pilot (supplement) attributes most of execute-and-derive to single-threaded Go-side ordinal consumption and witness merging. Multi-second latency thus suits batch, not interactive, agents. The bottleneck is single-threaded implementation, and replay buys database silence, zero charge, and audit identity, not time. A retained cost ablation and a naive-ledger comparison are in the supplement.

All 2,340 replay samples issued no business SQL and charged no Facts (row budget still consumed). Replay is no latency win: semantic replay exceeds direct re-execution in every cell; only idempotent replay at S2/SF10 undercuts it.

TABLE 1
Core workloads per cell (medians, ms, over 90 samples): rows, Facts, direct, novel, semantic and idempotent replay; overhead is novel/direct; –: no such cell.

Cell	Rows	Rel.	Dep.	Direct	Novel	Sem.	Idem.	Over.
S1/SF1	5,000	10,000	20,000	2.91	100.51	61.80	27.44	34.5×
S1/SF10	50,000	100,000	200,000	12.62	326.93	108.95	49.76	25.9×
S2/SF1	3	9	120,000	14.14	384.32	62.28	24.37	27.2×
S2/SF10	3	9	1,200,000	30.42	2459.32	62.65	24.12	80.8×
S3/10k-50k	10,000	10,000	50,000	2.43	151.06	62.34	28.65	62.1×
S3/1k-5k	1,000	1,000	5,000	0.83	97.15	56.61	25.01	116.9×
S3/45k-225k	45,000	45,000	225,000	8.67	319.42	88.91	43.67	36.9×
S4/depth-4	3	9	120,000	13.89	429.24	72.95	–	30.9×
S5/SF1	5,000	10,000	20,000	4.61	224.39	65.60	26.20	48.7×
S5/SF10	50,000	100,000	200,000	19.52	1431.39	112.37	50.19	73.3×
S6/100k-x16	100,000	1,600,000	1,700,000	173.89	3797.28	–	–	21.8×
S6/100k-x4	100,000	400,000	500,000	51.14	1133.42	–	–	22.2×
S6/100x16	100	1,600	1,700	0.90	129.82	–	–	144.0×
S6/100x4	100	400	500	0.76	85.20	–	–	112.0×
S6/10k-x16	10,000	160,000	170,000	16.28	586.97	–	–	36.0×
S6/10k-x4	10,000	40,000	50,000	5.33	201.72	–	–	37.8×

TABLE 2
Scale profile (RQ4): settlement and full-response p50 (ms) for one candidate against a same-size pre-seeded Dependency history.

History (Facts)	Settle, 0% ovl.	Settle, 100% ovl.	Response
10^4	40	–	–
1,035,000	163	124	1,306

Server phases per response are decomposed in the supplement. Because every descendant settles on one root head, a task tree's settlement throughput is bounded by that head (same-root contention and throughput: supplement). Table 2 answers RQ4 with the Scale profile (24 cells, 2,160 samples, every charge equal to the oracle's union minus history): settlement against a same-size history is sub-second at a million Facts and execute-and-derive dominates the response.

9.2 Adaptive Extraction and Provenance-Capture Cost

Scripted decompositions. The attack profile replays a frozen corpus (`483b667369a0`) of 7 sequences that try to obtain governed Facts twice (pages and an overlap around a complete query in both orders, four spellings of one predicate, a range split and its union, replays, a count-probe ladder with delegation; 135 samples, identical per-step charges per cell). Each decomposition was charged its complete query's Facts exactly once (12/18 for the six-step pagination in either order), each distinct page or range shape settled one composite, replays and rewrites charged nothing, and both set-operation variants failed closed. In the ladder, two count probes (one from a delegated child) and a detail primer consume the root's Outcome ceiling of 5. The child's third probe is refused with `EXPOSURE_BUDGET_EXHAUSTED`, so the agent learned 2 of 3 thresholds plus one refusal bit while the direct arm answered all three (supplement). The corpus is preregistered, and the adversary study above probes the strategy space. A simulated eight-point recipe sweep (supplement) first admits all 23 authorized statements and first recovers the secret at one grid point, $2\times$.

Adaptive trace against per-query control. The RLS profile replays one deterministic 100-query adaptive trace

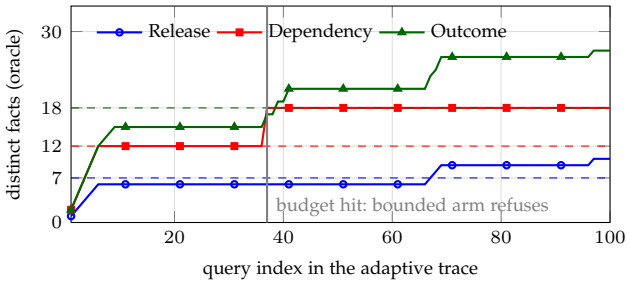


Fig. 2. Distinct governed facts by query index (trace-union oracle; solid) against the bounded arm's budgets (dashed); the bounded arm refuses at the first crossing.

(receipt look-ups, pages, equivalent predicate spellings, repeated aggregations, branches chosen from prior scalars) through three arms on one Product. The arms are PostgreSQL row-level security with a department policy and column grants, FactGate with an unlimited ledger, and FactGate bounded at $\lfloor 0.7U \rfloor$ of the trace's distinct facts $U = 10/18/27$ as counted by a separate trace-union oracle (budgets 7/12/18; 9 samples per arm, identical in every one). RLS answered all 100 queries (250 rows) in 16 ms. FactGate-unlimited answered all in 7.30 s, and its final ledger equals the oracle's union. FactGate-bounded answered 36 and refused query 37—the first whose oracle prefix exceeds a budget (Fig. 2)—with `EXPOSURE_BUDGET_EXHAUSTED`; it released nothing afterwards and ended at 6/12/15. A control step outside the policy rows and the column grant is refused at the column under both (supplement), so FactGate composes with per-query authorization. That RLS admits every query is definitional (no cumulative budget); the comparison shows where a set budget binds. Executed counter comparators (retained pilot) are in the supplement.

Provenance-capture cost. The ProvSQL profile (supplement) pairs byte-identical grouped SQL on pinned PostgreSQL 16.14, ProvSQL 1.11.0, and FactGate's full governed path: at 45k orders 47 ms, 90.9 s, and 2.62 s over 1,305,003 Dependency Facts, a boundary that is asymmetric by construction and ranks nothing.

9.3 Reproducibility Boundary

All measurements ran on one laptop-class WSL2 host with a digest-pinned PostgreSQL 16.14 image (supplement). The artifact is the repository at the cited commits (https://github.com/minmin-lab/factgate, with the evidence pack) and the retained raw evidence (216 files, 614 MB; Zenodo, doi:10.5281/zenodo.23149437; knowledge-product rounds and corpora, doi:10.5281/zenodo.23444544 and 10.5281/zenodo.23449501); every digest chain and macro is recomputable from it. Campaign numbers come from one submission commit (`faf4d095701b`), 33 fresh isolated deployments on one host, and three executions per profile whose per-execution novel medians differ by at most 5.1% of the pooled median (direct 10.5%, concurrency 7.0%); numbers marked pilot, retained, or simulated come from later runs, and the supplement maps each number to its class, run, and file. RQ1–RQ3 use small fixtures, the ordinal/bitmap evidence is warm, single-host, and TPC-H-derived (not official), and no result is generalized to cold-cache, multi-node, or production-SLO behavior.

10 Limitations and Threats to Validity

Semantic and subject scope. Outcome atoms and composites are governance units, not bits; FactGate omits rejected rows, non-output groups/pages, rank-only inputs, correlations, background knowledge, and physical reads, and provides neither minimal causality nor differential privacy (Section 3). **Query and identity scope.** Evidence uses frozen, manifest-proven publications. The closed fragment is a reporting-Product fragment, not ad-hoc analytics: the production lowerer admits 0 of the 22 TPC-H templates (first rejections: subqueries 7, arithmetic 5, multi-product `LIMIT`) and 2 of the 13 Star Schema Benchmark reporting queries (supplement tables). For LLM-written SQL from the Product sheet alone, 22 (55%) of 40 held-out questions written after the fragment froze lower unedited (admission, answers ungraded; 21–23 under a corrected sheet, pilot). Of the rest, 11 are engineering gaps, 6 need new rules, and 1 is agent output; they stop first at scalar functions, division or a subquery inside arithmetic, non-literal comparisons, `BETWEEN`/`LIKE`, `OR`, `LEFT JOIN`, or `LIMIT` over a join. Non-literal comparisons, `OR`, and outer joins need new rules (sketched in the supplement). The development set, adapted after its first pass, lowers 28 of 40 (70%) and measures iteration, not the frozen boundary. In a two-objective agent study (36 runs, supplement) an off-the-shelf agent completes the legitimate objective in every run after one refused listing, and its count-only probe stops at six probes. Stable keys and publications are trusted declarations, with View drift checked at request, approval, and execution.

Applicability envelope. Publication artifacts cost 620 B and 22.8 μs per Fact (RQ5 Day 0). A 10^9-Fact Product would need hundreds of gigabytes and hours per build, so the envelope is 10^5–10^6-row Products; a retained pilot ladder settles 10,500,000 Dependency facts in one query (supplement). Same-root settlement serializes on the head (Section 9).

Knowledge Products. The evaluation uses synthetic corpora and scripted agents; extraction-rule coverage bounds value-level accounting, paraphrases are charged separately, ranking leakage is unquantified, and an approximate index would make an exact replay a new composite (supplement). **Governance and deployment scope.** Profile correctness is trusted—FactGate validates bindings, semantics, and settlement, not policy appropriateness—and the single database path and demo approval adapter offer no production integration study [8]. The guarantee composes with controls it does not supply: task-tree and refusal-rate caps (Proposition 6), timing padding, and root expiry; splitting one objective across independently approved roots is fresh capacity no ledger spans, so its defence is each root's approval, not accounting.

11 Conclusion

FactGate provides profile-relative exact accounting of Result, Dependency, and Outcome sets within one root family behind one scoped claim: no governed artifact becomes `AVAILABLE` before atomic settlement.

REFERENCES

[1] C. Summers, H. Mohammed, and E. Wu, "Please don't kill my vibe: Empowering agents with data flow control," in *Proc. CIDR (CIDR 2026)*, 2026.

[2] C. Summers and E. Wu, "Data flow control: Data safety policies for AI agents," arXiv preprint arXiv:2606.05679, 2026.

[3] P. Koutris, P. Upadhyaya, M. Balazinska, B. Howe, and D. Suciu, "Query-based data pricing," in *Proc. ACM PODS*, 2012, pp. 167–178.

[4] S. Deep and P. Koutris, "QIRANA: A framework for scalable query pricing," in *Proc. ACM SIGMOD*, 2017.

[5] H. Peng, X. Miao, Y. Fu, J. Zhang, S. Deng, and J. Yin, "On scalable query pricing in data marketplaces," in *Proc. IEEE ICDE*, 2025.

[6] T. J. Green, G. Karvounarakis, and V. Tannen, "Provenance semirings," in *Proc. ACM PODS*, 2007, pp. 31–40.

[7] Y. Amsterdamer, D. Deutch, and V. Tannen, "Provenance for aggregate queries," in *Proc. ACM PODS*, 2011, pp. 153–164.

[8] M. Tonnarelli, F. Scaramuzza, S. Harrer, and L. W. Dietz, "Data product MCP: Chat with your enterprise data," arXiv preprint arXiv:2601.08687, 2026.

[9] L. W. Dietz, A. Wider, and S. Harrer, "Automating data governance with generative AI," *Proc. AAAI/ACM AIES*, vol. 8, no. 1, pp. 760–771, 2025.

[10] Bitol, "Open data contract standard (ODCS), version 3.0.1," LF AI & Data Foundation, 2025.

[11] R. Agrawal, J. Kiernan, R. Srikant, and Y. Xu, "Auditing compliance with a hippocratic database," in *Proc. VLDB*, 2004, pp. 516–527.

[12] R. Motwani, S. U. Nabar, and D. Thomas, "Auditing SQL queries," in *Proc. IEEE ICDE*, 2008, pp. 287–296.

[13] K. Kenthapadi, N. Mishra, and K. Nissim, "Simulatable auditing," in *Proc. ACM PODS*, 2005, pp. 118–127.

[14] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*. Now Publishers, 2014.

[15] M. Mazmudar, T. Humphries, J. Liu, M. Rafuse, and X. He, "Cache me if you can: Accuracy-aware inference engine for differentially private data exploration," *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 574–586, 2022.

[16] S. Zhang and X. He, "DProvDB: Differentially private query processing with multi-analyst provenance," 2023.

[17] D. Dobkin, A. K. Jones, and R. J. Lipton, "Secure databases: Protection against user influence," *ACM Trans. Database Syst.*, vol. 4, no. 1, pp. 97–106, 1979.

[18] D. E. Denning and J. Schlörer, "A fast procedure for finding a tracker in a statistical database," *ACM Transactions on Database Systems*, vol. 5, no. 1, pp. 88–102, 1980.

[19] S. U. Nabar, B. Marthi, K. Kenthapadi, N. Mishra, and R. Motwani, "Towards robustness in query auditing," in *Proc. VLDB*, 2006, pp. 151–162.

[20] F. D. McSherry, "Privacy integrated queries: An extensible platform for privacy-preserving data analysis," in *Proc. ACM SIGMOD*, 2009, pp. 19–30.

[21] P. Buneman, S. Khanna, and W.-C. Tan, "Why and where: A characterization of data provenance," in *Database Theory—ICDT 2001*, ser. Lecture Notes in Computer Science, vol. 1973, 2001, pp. 316–330.

[22] J. Cheney, L. Chiticariu, and W.-C. Tan, "Provenance in databases: Why, how, and where," *Foundations and Trends in Databases*, vol. 1, no. 4, pp. 379–474, 2009.

[23] B. S. Arab, S. Feng, B. Glavic, S. Lee, X. Niu, and Q. Zeng, "GProM—a swiss army knife for your provenance needs," *IEEE Data Engineering Bulletin*, vol. 41, no. 1, pp. 51–62, 2018.

[24] P. Senellart, L. Jachiet, S. Maniu, and Y. Ramusat, "ProvSQL: Provenance and probability management in PostgreSQL," *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 2034–2037, 2018.

[25] A. Sen, S. Maniu, and P. Senellart, "ProvSQL: A general system for keeping track of the provenance and probability of data," in *Proc. IEEE ICDE*, Montréal, Canada, May 2026, arXiv:2504.12058.

[26] R. K. Thomas and R. S. Sandhu, "Task-based authorization controls (TBAC): A family of models for active and enterprise-oriented authorization management," in *Proc. IFIP WG 11.3 Workshop on Database Security*, 1997, pp. 166–181.

[27] J.-W. Byun and N. Li, "Purpose based access control for privacy protection in relational database systems," *VLDB J.*, vol. 17, no. 4, pp. 603–619, 2008.

[28] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud," in *Proc. NDSS*, 2014.

[29] S. Rizvi, A. Mendelzon, S. Sudarshan, and P. Roy, "Extending query rewriting techniques for fine-grained access control," in *Proc. ACM SIGMOD*, 2004, pp. 551–562.

[30] J. Park and R. Sandhu, "The UCONABC usage control model," *ACM Trans. Inf. Syst. Secur.*, vol. 7, no. 1, pp. 128–174, 2004.

[31] M. Abadi and C. Fournet, "Access control based on execution history," in *Proc. NDSS*, 2003.

[32] N. Li, B. N. Grosof, and J. Feigenbaum, "Delegation logic: A logic-based approach to distributed authorization," *ACM Trans. Inf. Syst. Secur.*, vol. 6, no. 1, pp. 128–171, 2003.

[33] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation: Row Security Policies*, 2026.

[34] A. Mehta, E. Elnikety, K. Harvey, D. Garg, and P. Druschel, "Qapla: Policy compliance for database-backed systems," in *Proc. USENIX Security*, 2017, pp. 1463–1479.

[35] W. Zhang, E. Sheng, M. Chang, A. Panda, M. Sagiv, and S. Shenker, "Blockaid: Data access policy enforcement for web applications," in *Proc. USENIX OSDI*, 2022, pp. 1201–1219.

[36] Model Context Protocol Contributors, *Model Context Protocol Specification, 2025-11-25: Authorization and Tools*, 2025.

[37] T. South *et al.*, "Position: AI agents need authenticated delegation," in *Proc. ICML*, 2025, pp. 82 211–82 231.

[38] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Proc. NeurIPS*, 2024.

[39] R. Liu, X. Chen, J. Zhang, Q. Zhang, Y. Zhang, and B. Yang, "SafeNLIDB: A privacy-preserving safety alignment framework for LLM-based natural language database interfaces," in *Proc. AAAI*, vol. 40, no. 38, 2026, pp. 32 123–32 131.

[40] J. Park, D. Nguyen, and R. Sandhu, "A provenance-based access control model," in *Proc. PST*, 2012, pp. 137–144.

[41] E. Debenedetti *et al.*, "Defeating prompt injections by design," arXiv preprint arXiv:2503.18813, 2025.

[42] R. Stanley, A. Verma, L. Tsai, K. Kallas, and S. Kumar, "An AI agent execution environment to safeguard user data," *arXiv preprint arXiv:2604.19657*, 2026.

[43] S. S. Bhatt, T. Rajore *et al.*, "Enterprise AI must enforce participant-aware access control," 2025, arXiv:2509.14608, September 2025.

[44] Anonymous, "If agents were angels, no governance would be necessary: Out-of-band policy enforcement at a trusted tool boundary," 2026, arXiv:2608.27646, August 2026.

[45] Y. Wang *et al.*, "Silent leaks: Implicit knowledge extraction attack on RAG systems through benign queries," 2025, arXiv:2505.15420, May 2025.

[46] Y. He, Y. Chen *et al.*, "External data extraction attacks against retrieval-augmented large language models," 2025, arXiv:2510.02964, October 2025.

[47] J. Xie and S. Li, "OCELOT: Inference-leakage budgets for privacy-preserving LLM agents," 2026, arXiv:2606.12341, June 2026.

[48] K. D. Bowers, L. Bauer, D. Garg, F. Pfenning, and M. K. Reiter, "Consumable credentials in linear-logic-based access-control systems," in *Proc. NDSS*, 2007.

[49] K. T. Pollack, D. D. E. Long, R. A. Golding, A. Becker-Szendy, and B. Reed, "Quota enforcement for high-performance distributed storage systems," in *Proc. IEEE MSST*, 2007, pp. 72–84.

[50] Y. Cui and J. Widom, "Lineage tracing for general data warehouse transformations," in *Proc. VLDB*, 2001, pp. 471–480.

[51] S. Chambi, D. Lemire, O. Kaser, and R. Godin, "Better bitmap performance with Roaring bitmaps," *Software: Practice and Experience*, vol. 46, no. 5, pp. 709–719, 2016.

[52] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology—CRYPTO '87*, ser. Lecture Notes in Computer Science, vol. 293, 1987, pp. 369–378.

[53] M. P. Herlihy and J. M. Wing, "Linearizability: A correctness condition for concurrent objects," *ACM Trans. Program. Lang. Syst.*, vol. 12, no. 3, pp. 463–492, 1990.

[54] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[55] R. J. Hyndman and Y. Fan, "Sample quantiles in statistical packages," *The American Statistician*, vol. 50, no. 4, pp. 361–365, 1996.